

Async Research Assistant

Shahin Safarli · Jeyhuna Sevdieva · Emil Jafarov

2026-05-23

Overview

01 Problem & Motivation

Why async research matters

02 System Architecture

High-level design overview

03 AI Layer

Fetchers, providers, synthesizer

04 Concurrency Pipeline

asyncio.gather + graceful degradation

05 SE Layer Components

Config · Cache · CLI · Storage · Docker

06 Performance Benchmarks

Sequential vs concurrent timing

07 Testing & Coverage

81 tests · 86% coverage

08 Team Contributions

PRs, roles, git workflow

09 Demo & Live Output

Offline demo run results

10 Summary & Takeaways

What we built and learned

The Research Question Problem

A user poses a research question. The system must query Wikipedia, arXiv, and a web-search API, then synthesise a single cited answer.



Sequential Latency

3 sources × ~400 ms each
= ~1.2 s minimum wait time



No Fault Isolation

One failed source crashes
the entire pipeline

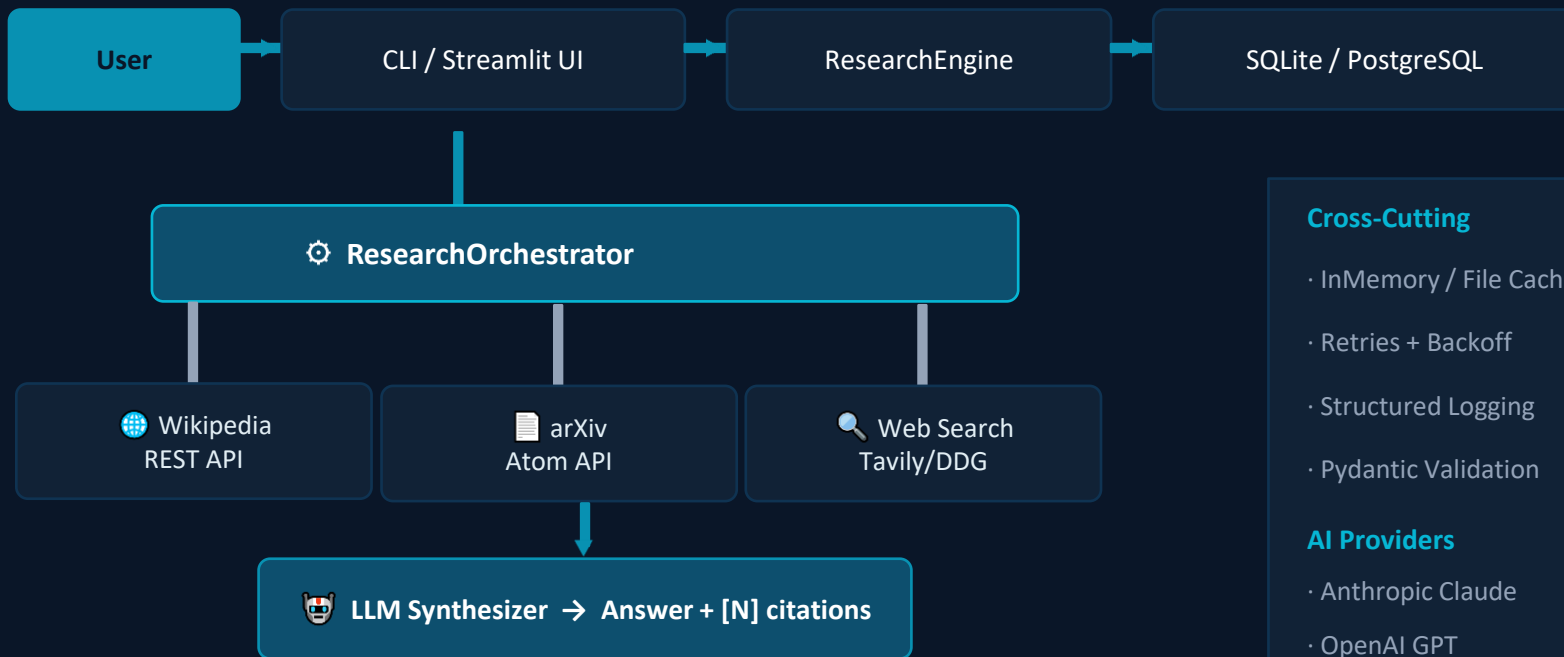


Repeated API Calls

Same query hits remote APIs
on every single request

→ **Solution: `asyncio.gather` + TTL caching + graceful degradation**

End-to-End Design



Cross-Cutting

- InMemory / File Cache
- Retries + Backoff
- Structured Logging
- Pydantic Validation

AI Providers

- Anthropic Claude
- OpenAI GPT
- Google Gemini

The Provided ai/ Module



Source Fetchers

- `fetch_wikipedia(query)`
- `fetch_arxiv(query)`
- `fetch_web(query)`
- Shared `httpx.AsyncClient`
- Returns `list[Source]`



LLM Synthesizer

- `synthesize(question, sources)`
- Produces 3–6 sentence answer
- Inline [N] citations
- Drops hallucinated indices
- Returns `AnswerWithCitations`

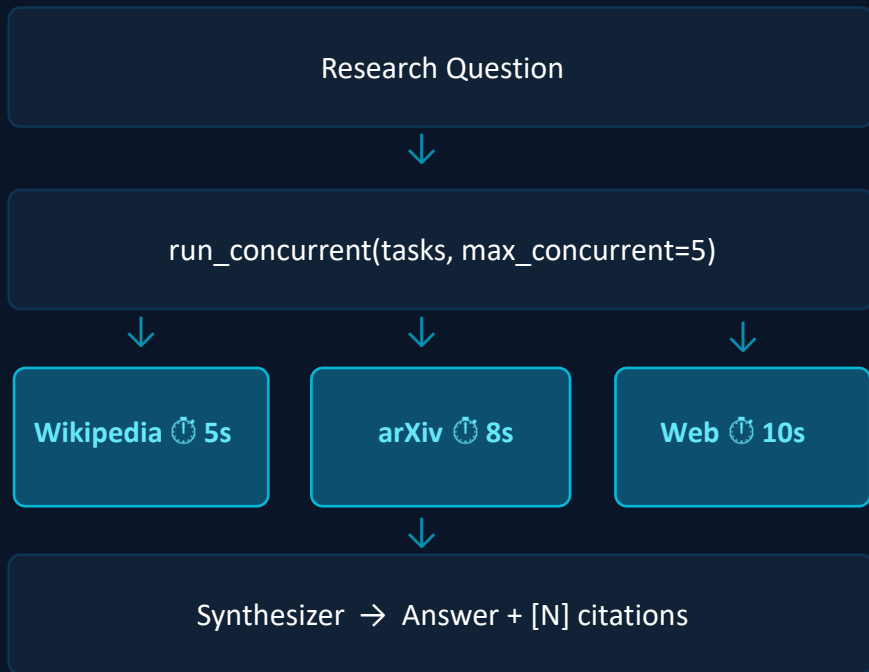


Provider Abstraction

- Anthropic / OpenAI / Gemini
- Selected via env vars
- `LLM_PROVIDER=anthropic`
- `EMBEDDING_PROVIDER=openai`
- Zero code changes to swap

🚫 Contract: *ai/* is read-only — SE logic calls through *ai.fetch_wikipedia*, *ai.fetch_arxiv*, *ai.fetch_web*, *ai.synthesize*

asyncio.gather with Per-Source Timeouts



asyncio.gather

All three sources fire simultaneously inside a semaphore-bounded gather



Graceful Degradation

`return_exceptions=True` — one timeout won't kill the whole answer



Connection Reuse

Single `httpx.AsyncClient` shared across all three fetchers per request



arXiv Web Fallback

If Atom returns empty, falls back to `site:arxiv.org` web search

What the Team Built

config.py

pydantic-settings, env-driven typed config, all provider keys + timeouts

models.py

QuestionRequest ·
ResearchResult ·
CitationRecord — full
Pydantic validation

cache.py

InMemoryCache +
FilesystemCache, TTL-
aware (source, query),
canonicalized keys

ai_service.py

Tenacity retry + backoff,
per-call timeouts, rate-
limit wrapper

researcher.py

ResearchEngine — Fetch
→ Synthesize → Persist,
history via repository

cli.py

Click CLI: ask · history, --
sources · --no-cache
flags, citation rendering

repository.py

BaseRepository ABC,
SQLiteRepository
(aiosqlite),
PostgreSQLRepository
(asyncpg)

Dockerfile + docker- compose

Multi-stage build,
postgres:16-alpine
service, healthcheck +
wait-for-db

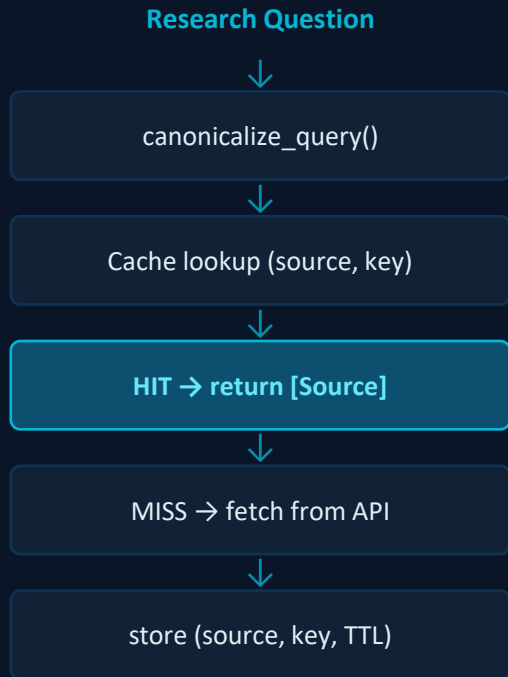
app.py (Streamlit)

Bonus web UI — ask
questions interactively,
history sidebar

CI (GitHub Actions)

ruff lint + mypy types,
pytest + coverage gate,
docker build check

TTL-Aware Source Cache



InMemoryCache

- ▶ Python dict + timestamps
- ▶ TTL per entry (default 300 s)
- ▶ Configurable via settings
- ▶ Thread-safe for asyncio

FileSystemCache

- ▶ JSON files, keyed by hash
- ▶ Persists across restarts
- ▶ --no-cache bypasses both
- ▶ ABC for easy extension

Cache Key Construction

```
def canonicalize_query(q):  
    q = q.lower().strip()  
    q = re.sub(r"\s+", " ", q)  
    q = q.rstrip("?!.")  
    return q
```

```
key = (source, canonical)
```

Same result for:

"WHAT IS PHOTOSYNTHESIS?"

"what is photosynthesis"

"What is photosynthesis?"

Sequential vs Concurrent — Consistent 3× Speedup

3.0×

Speedup (all runs)

0.50s

Concurrent (5 Qs)

1.50s

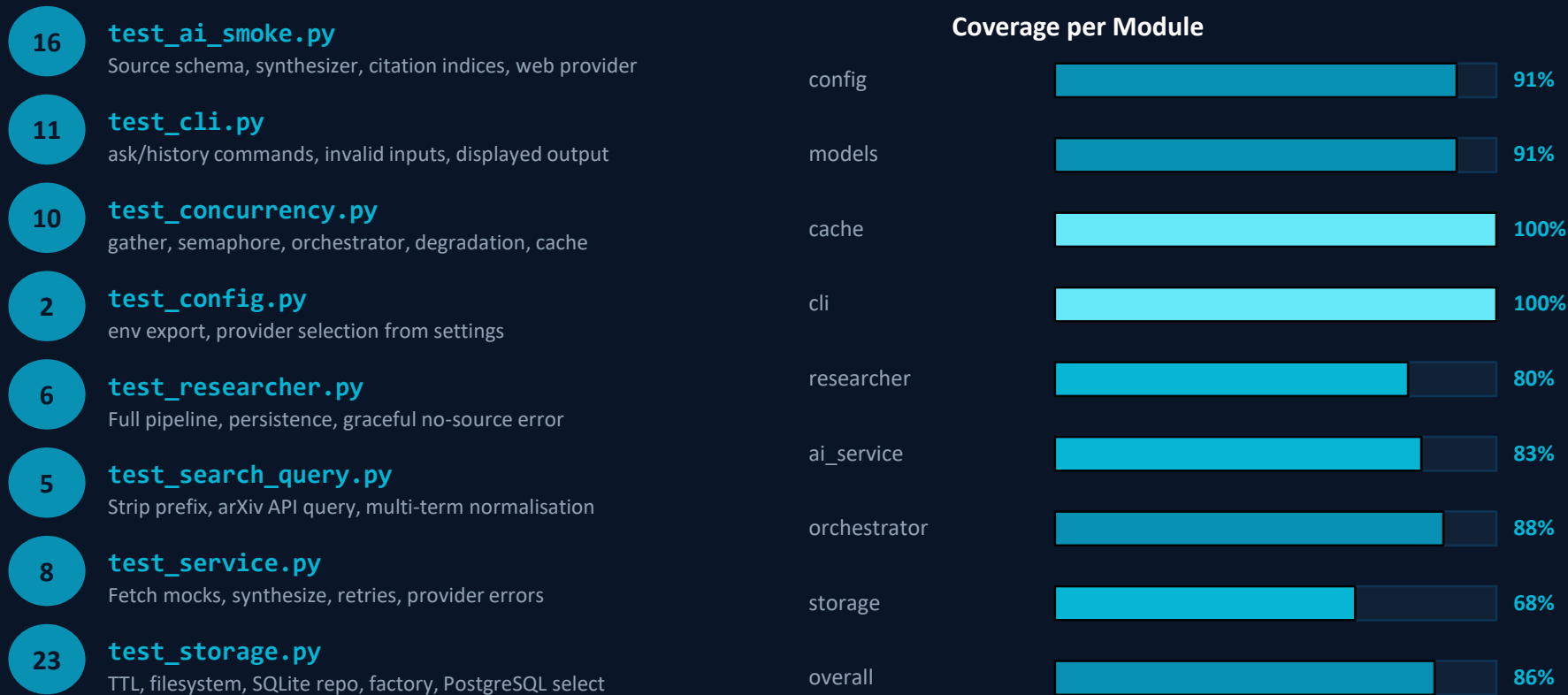
Sequential (5 Qs)

86%

Test Coverage

Network Simulation	Sequential (s)	Concurrent (s)	Speedup
Fast (delay=0.05 s)	0.753	0.252	3.0×
Typical (delay=0.10 s)	1.505	0.502	3.0×
Slow (delay=0.15 s)	2.255	0.752	3.0×
Large batch N=10	1.504	0.502	3.0×

81 Tests Passed · 86% Coverage



Who Built What



Shahin Safarli

@shahinsafarli

~40%

- `src/config.py` — pydantic-settings, env sync
- `src/cli/` — Click CLI (ask, history)
- `tests/` — unified offline test framework (81 tests)
- `src/storage/postgres_repository.py` — PostgreSQL backend
- Project scaffolding, .gitignore, PR template
- PRs: #1, #7, #9, #12



Emil Jafarov

@Emil-Jafarov-06

~30%

- `src/services/ai_service.py` — retry/timeout wrapper
- `src/concurrency/` — orchestrator + pipeline
- `app/` — Streamlit web UI + Docker setup
- `README.md` — project overview & setup guide
- `docs/report` — full project report
- PRs: #3, #6, #11, #13



Jeyhuna Sevdieva

@Jeyhunaa

~30%

- `src/storage/` — SQLite repository (session persistence)
- `src/services/cache_service.py` — TTL cache
- `src/utils/search_query.py` + `bench.py`
- `docs/architecture.md` + `demo.py`
- Presentation slides + `CONTRIBUTION_STATEMENT.md`
- PRs: #2, #4, #5, #8, #14

PR-Based Development — 14 Pull Requests

PR	Branch	Author	Files Changed
#1	shahin/config-and-models	Shahin	config.py + models.py
#2	jeyhuna/architecture-docs	Jeyhuna	docs/architecture.md
#3	emil/ai-service-and-core	Emil	ai_service.py + researcher.py
#4	jeyhuna/storage-layer	Jeyhuna	repository.py (SQLite)
#5	jeyhuna/cache-and-search-utils	Jeyhuna	cache.py + search_query.py
#6	emil/concurrency-and-researcher	Emil	orchestrator.py + pipeline.py
#7	shahin/cli-and-entry-point	Shahin	cli.py + __main__.py
#8	jeyhuna/demo-script	Jeyhuna	scripts/demo.py + bench.py
#9	shahin/combined-testing-suite	Shahin	All 8 test_*.py files
#10	emil/streamlit-and-docker	Emil	app.py + Dockerfile + docker-compose
#11	emil/concurrency-and-core	Emil	Orchestrator async core
#12	shahin/postgres-storage	Shahin	postgres_repository.py + asyncpg
#13	emil/readme-and-report	Emil	README.md + docs/report
#14	jeyhuna/presentation-and-contribution	Jeyhuna	Presentation + CONTRIBUTION_STATEMENT.md

Rule: every PR reviewed by a teammate · Never push direct to main · Tag v1.0-final after PR #14

Three Deployment Modes



Local / SQLite

Fastest to start

```
pip install -r requirements.txt
python -m researcher ask "..."
```

No Docker needed. SQLiteRepository auto-selected.



Docker + SQLite

Portable & offline

```
docker build -t researcher .
docker run --env-file .env
researcher
```

Single container. Supports offline demo mode.



Docker Compose + PostgreSQL

Production mode

```
docker compose up --build
# postgres:16-alpine + app
```

PostgreSQLRepository auto-selected via DATABASE_URL.

```
get_repository() factory: sqlite+aiosqlite:// → SQLiteRepository | postgresql+asyncpg:// → PostgreSQLRepository
```

Offline Demo — 5 Research Questions

Q1

What is photosynthesis and what are its main stages?

~3.5 s

[1] Wikipedia [2] arXiv [3] Web · elapsed ~3.5 s · sources: 3 · failed: none

Q2

How do transformer-based language models handle long context windows?

~3.5 s

[1] Wikipedia [2] arXiv [3] Web · elapsed ~3.5 s · sources: 3 · failed: none

Q3

What were the main causes of the 2008 financial crisis?

~3.5 s

[1] Wikipedia [2] arXiv [3] Web · elapsed ~3.5 s · sources: 3 · failed: none

Q4

What is the current state of fusion energy research?

~3.5 s

[1] Wikipedia [2] arXiv [3] Web · elapsed ~3.5 s · sources: 3 · failed: none

Q5

How does CRISPR-Cas9 gene editing work at a molecular level?

~3.5 s

[1] Wikipedia [2] arXiv [3] Web · elapsed ~3.5 s · sources: 3 · failed: none

What We Built & What We Learned

Deliverables Completed

- ✓ `asyncio.gather` with per-source timeouts
- ✓ Graceful degradation (`return_exceptions=True`)
- ✓ InMemory + Filesystem TTL cache
- ✓ Click CLI: `ask` / `history` commands
- ✓ Pydantic validation at every layer
- ✓ Exponential backoff via tenacity
- ✓ SQLite + PostgreSQL dual-backend
- ✓ `docker-compose` with `postgres:16`
- ✓ Streamlit web UI (bonus)
- ✓ GitHub Actions CI (lint + test + build)
- ✓ 81 tests · 86% coverage
- ✓ LaTeX report + slides + artefacts

Key Engineering Lessons

◆ **asyncio is not a silver bullet**

`gather` helps, but timeouts per-task are essential to avoid head-of-line blocking.

◆ **Cache keys need canonicalisation**

Upper/lower, trailing punctuation — small differences bypass caches silently.

◆ **Composition > Inheritance**

Injecting `ai_svc`, `cache`, `repo` makes `ResearchEngine` fully testable offline.

◆ **Mock the boundary, not the logic**

`respx` + `pytest-httpx` mock HTTP; the orchestrator logic itself runs for real.

Thank You

Questions & Discussion

Shahin Safarli

@shahinsafarli

Jeyhuna Sevdiev

@Jeyhunaa

Emil Jafarov

@Emil-Jafarov-06

github.com/shahinsafarli/async-research-assistant

tag: v1.0-final · Topic 4 — AIENG Final Project · 2026-05-23